VISUAL SPEAKER NOTES • CONTENT-DRIVEN EDITION

How Senior Engineers Build With AI

How Senior Engineers Build With AI (Full Intercom Clone)

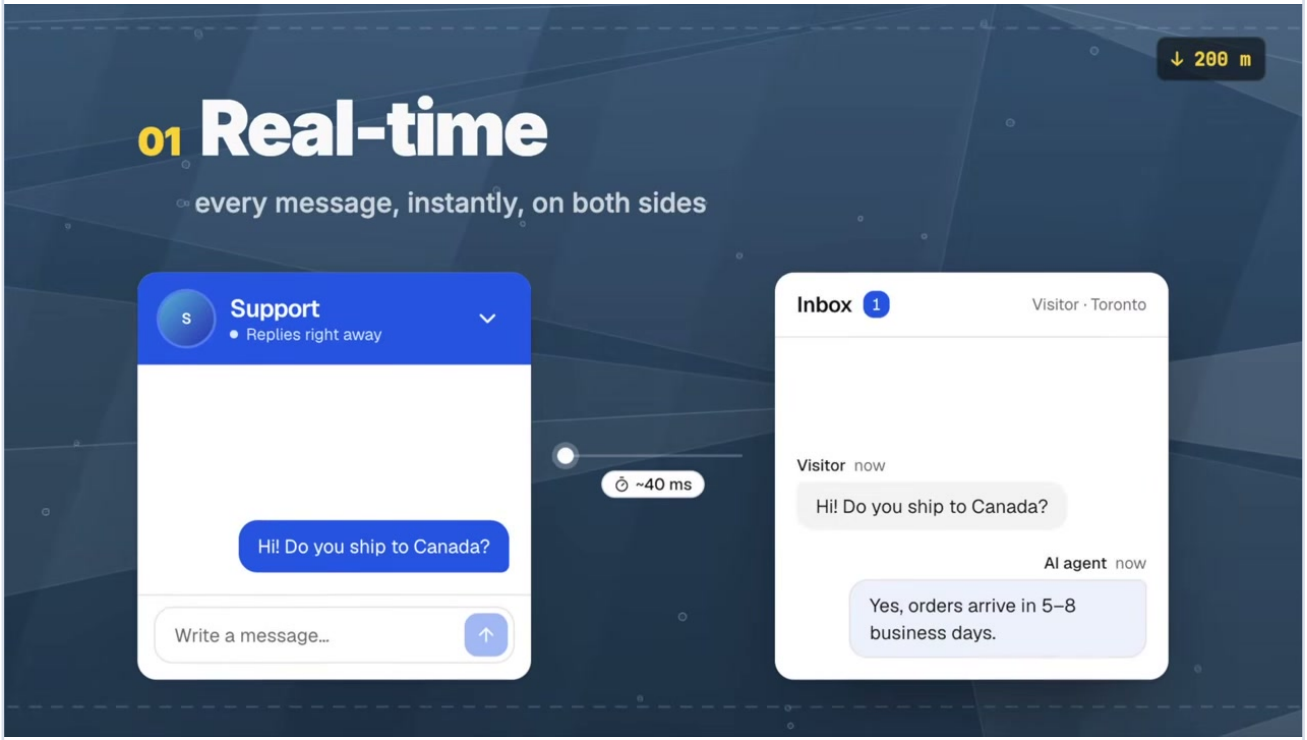
Jan Marshal • 2026-10-04 • 5:55:23

- 25 substantive visual(s), selected by content
- Repeated builds, presenter-only shots, and filler excluded
- Output length expands or contracts with the source material
- Speaker claims and caveats belong in the notes

https://www.youtube.com/watch?v=H84vfy96m_k

Intro

Why a support bubble is a systems problem



SPEAKER NOTES

- An Intercom-style widget looks simple, but production behavior spans real-time messaging, RAG, authentication, tenant isolation, storage, background jobs, and deployment.
- A broad prompt can generate thousands of plausible lines while leaving the application structurally broken; the failure is insufficient planning, not simply a weak model.
- Marshall Desk is introduced as the end-to-end target: business onboarding, configurable widget, parallel document ingestion, grounded answers, human handoff, and an operator inbox.
- The central claim is that senior AI-assisted engineering is a workflow of scoping, architecture, context, verification, and review.

WHY IT MATTERS

Decompose the visible feature into its hidden systems before asking an agent to build it.

Prerequisites: ADEs, Skills & Models

Reduce interaction friction before implementation



SPEAKER NOTES

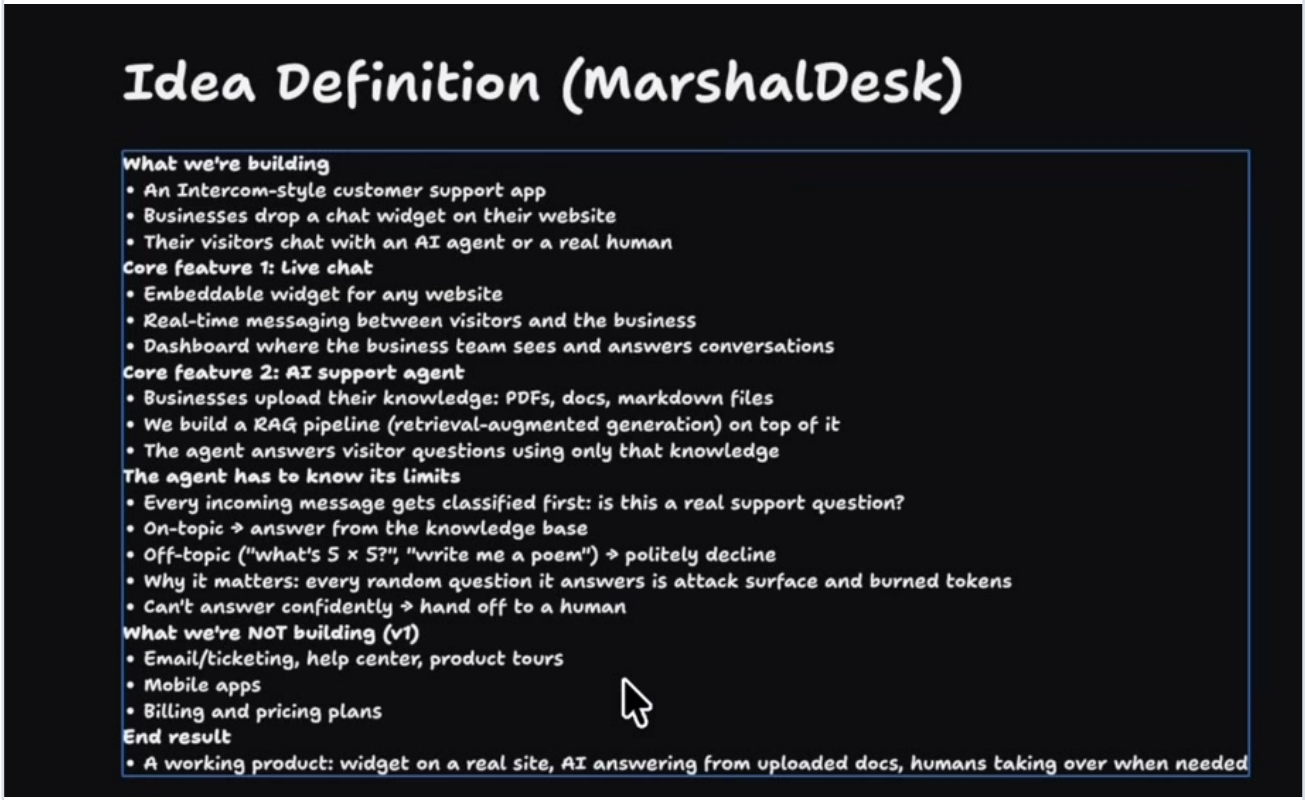
- Choose a fluent ADE—Cursor, Codex, Claude Code, or an equivalent harness—instead of constantly switching tools.
- Browser access is essential: the agent must inspect and verify the product, not merely compile code.
- Keep skills focused and use voice for richer prompts, then review transcription and ambiguity before execution.

WHY IT MATTERS

Reduce interaction friction and give the agent a way to see its own output.

Defining the Idea

Specify V1 and its exclusions



SPEAKER NOTES

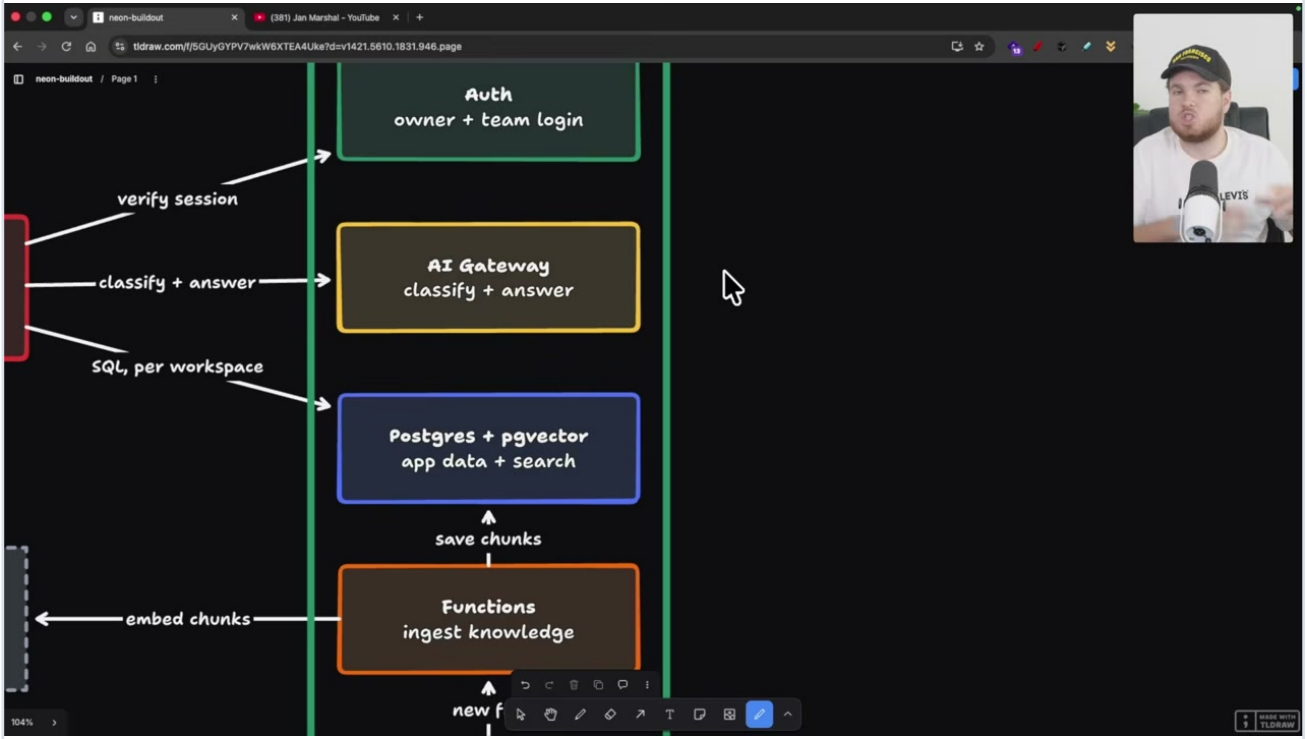
- V1 combines an embeddable real-time widget with an AI agent grounded in business-owned documents.
- Off-topic or unsupported requests are declined or handed to a person rather than answered from imagination.
- Email, ticketing, help center, mobile, billing, and pricing are excluded; success is a working grounded widget with handoff.

WHY IT MATTERS

Explicit exclusions keep V1 achievable and prevent silent scope expansion.

System Architecture

Human-owned technical boundaries



SPEAKER NOTES

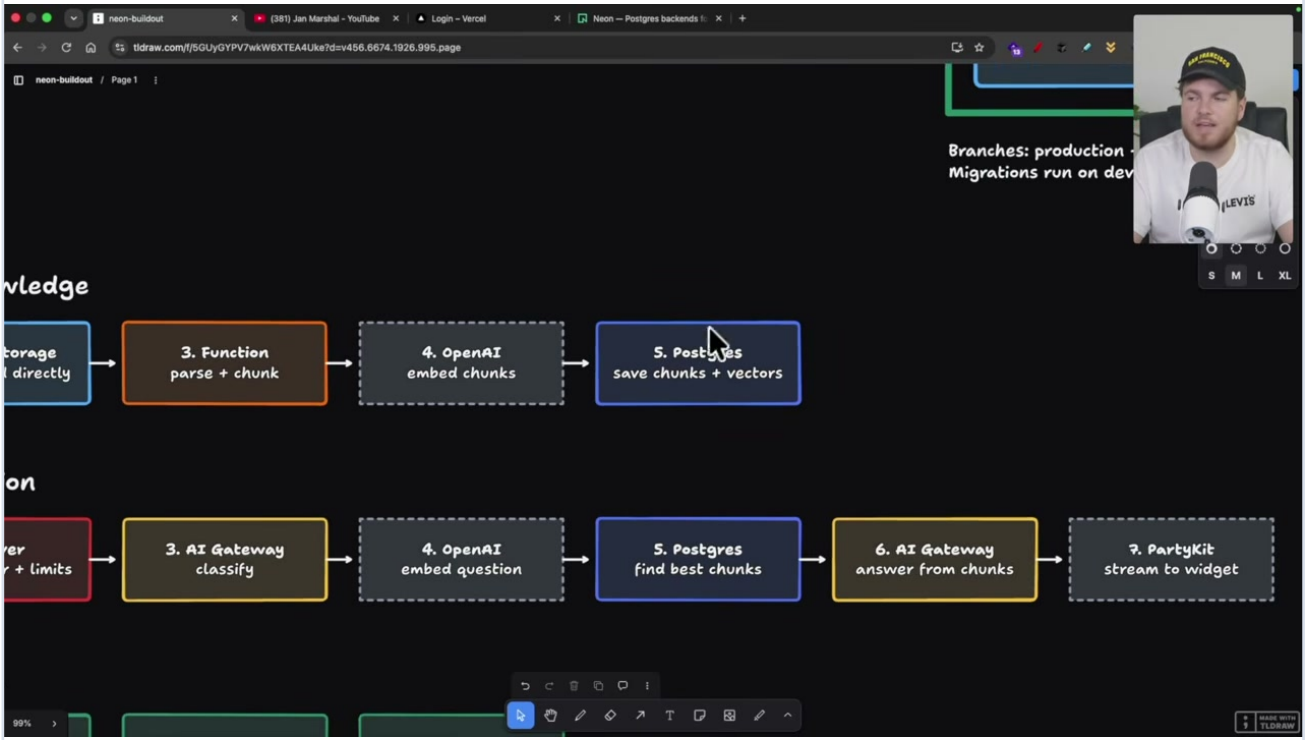
- The developer chooses the architecture; the coding agent should not silently decide framework, tenancy, persistence, or deployment boundaries.
- Next.js serves the dashboard and application layer, while an iframe isolates the customer widget from the host website.
- Neon consolidates PostgreSQL, pgvector, authentication, object storage, functions, and the AI gateway; PartyKit/PartyServer supplies real-time messaging.
- Long-running document ingestion belongs in background functions, and large uploads should go directly to private storage through authorized presigned URLs.

WHY IT MATTERS

Architecture should remove consequential ambiguity without becoming an endless technology comparison exercise.

The 3 Core Application Flows

Design around user journeys, not isolated screens



SPEAKER NOTES

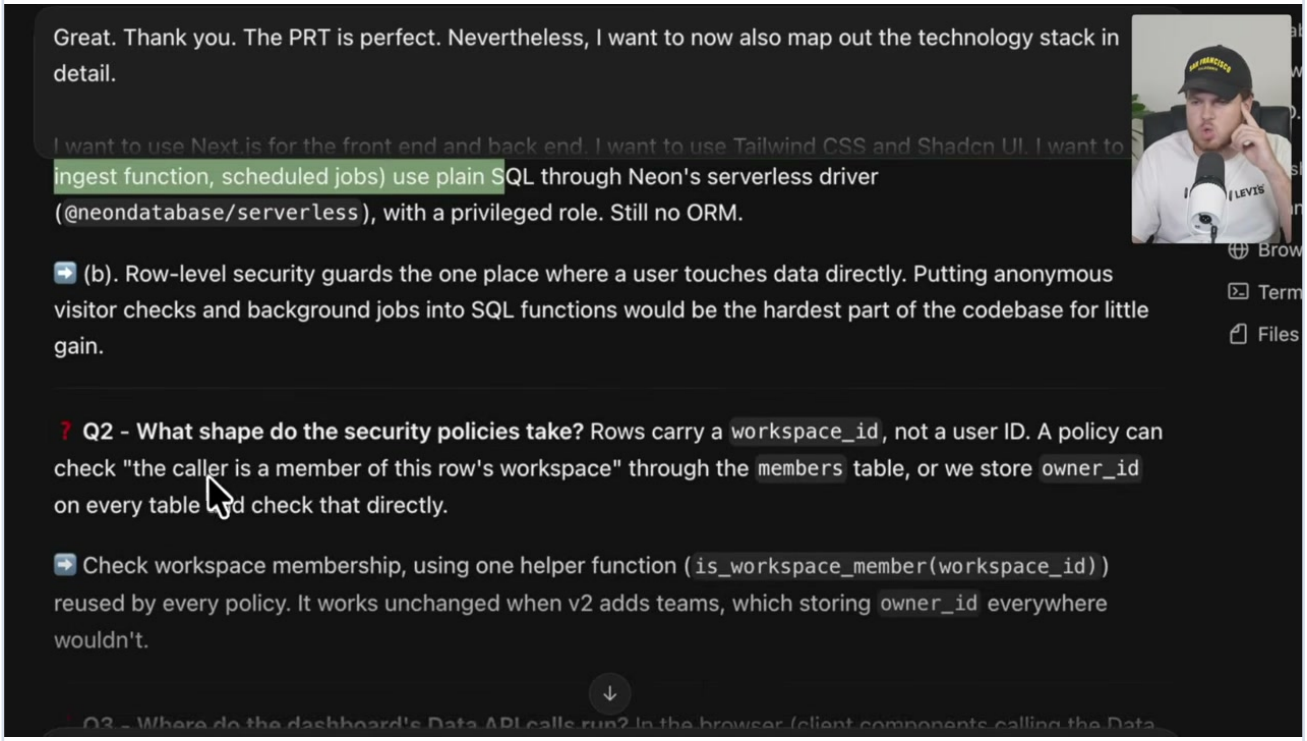
- The product is organized around business setup, visitor conversation, and operator takeover rather than a disconnected list of pages.
- Business setup covers authentication, workspace creation, agent customization, allowed domains, embed installation, and knowledge uploads.
- Visitor flow covers identity, conversation creation, real-time exchange, grounded AI answers, and escalation when confidence is insufficient.
- Operator flow covers notifications, inbox state, visitor context, manual replies, and conversation lifecycle actions.

WHY IT MATTERS

End-to-end flows reveal state transitions and security boundaries that screen-by-screen prompting misses.

Writing the PRD & Choosing the Tech Stack

Interrogate the specification before generating code



SPEAKER NOTES

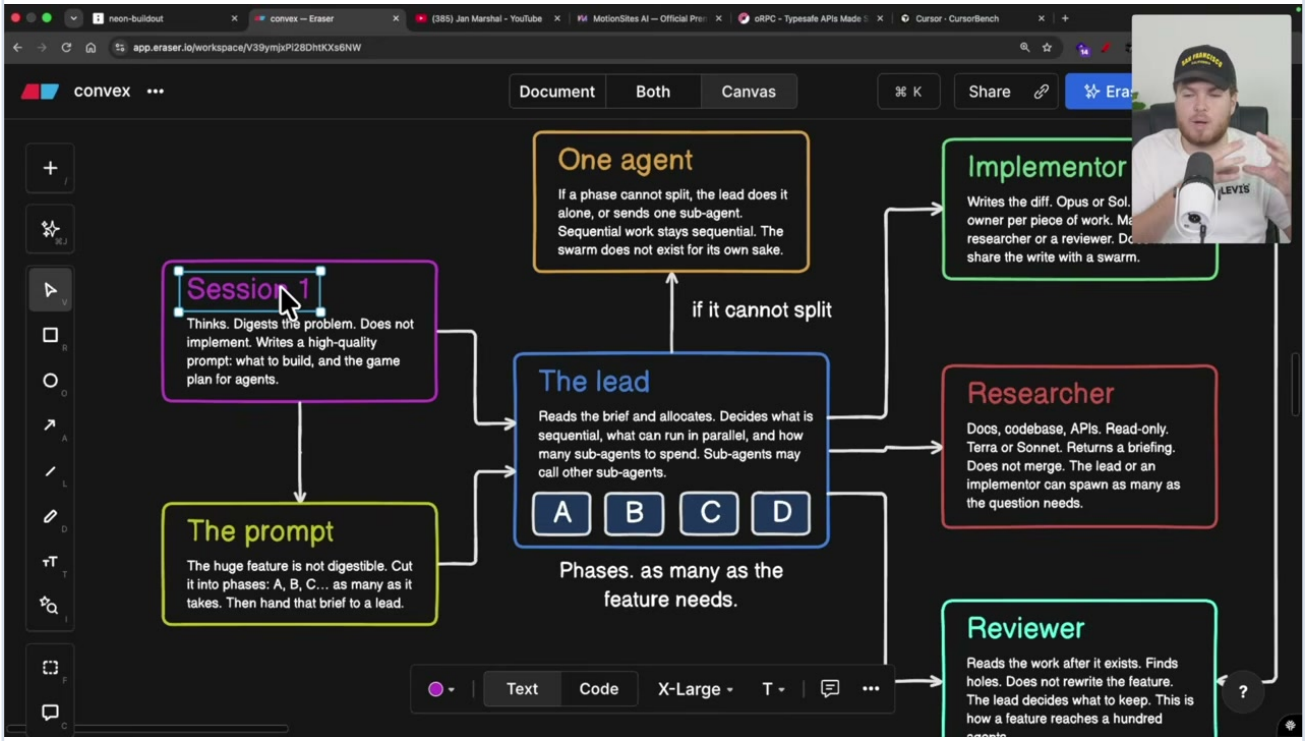
- The initial PRD is only a draft; the agent repeatedly asks questions and updates it until major assumptions, states, and edge cases become explicit.
- A dedicated stack document records where each technology runs, why it was chosen, and how authentication, authorization, real time, and deployment interact.
- The design moves from a mixed Data API/RLS approach to trusted server-side Prisma because one query model, shared types, and migrations are easier to review.
- oRPC and TanStack Query provide typed contracts, server/client data flow, mutations, and cache updates from real-time events.

WHY IT MATTERS

Generate, interrogate, and lock the stack before implementation so the agent does not invent incompatible foundations.

My AI Coding Workflow

Models, prompts, roles, and fresh context



SPEAKER NOTES

- One session develops shared understanding through discussion; it then compiles the decisions into a precise implementation prompt for a fresh session.
- A lead agent plans and delegates, read-only researchers gather current evidence, implementers write code, and reviewers inspect the completed work.
- Different models are selected for capability, latency, and cost rather than assigning the most expensive model to every role.
- Subagents protect the lead's context window and allow parallel work, but the human still performs a final review.

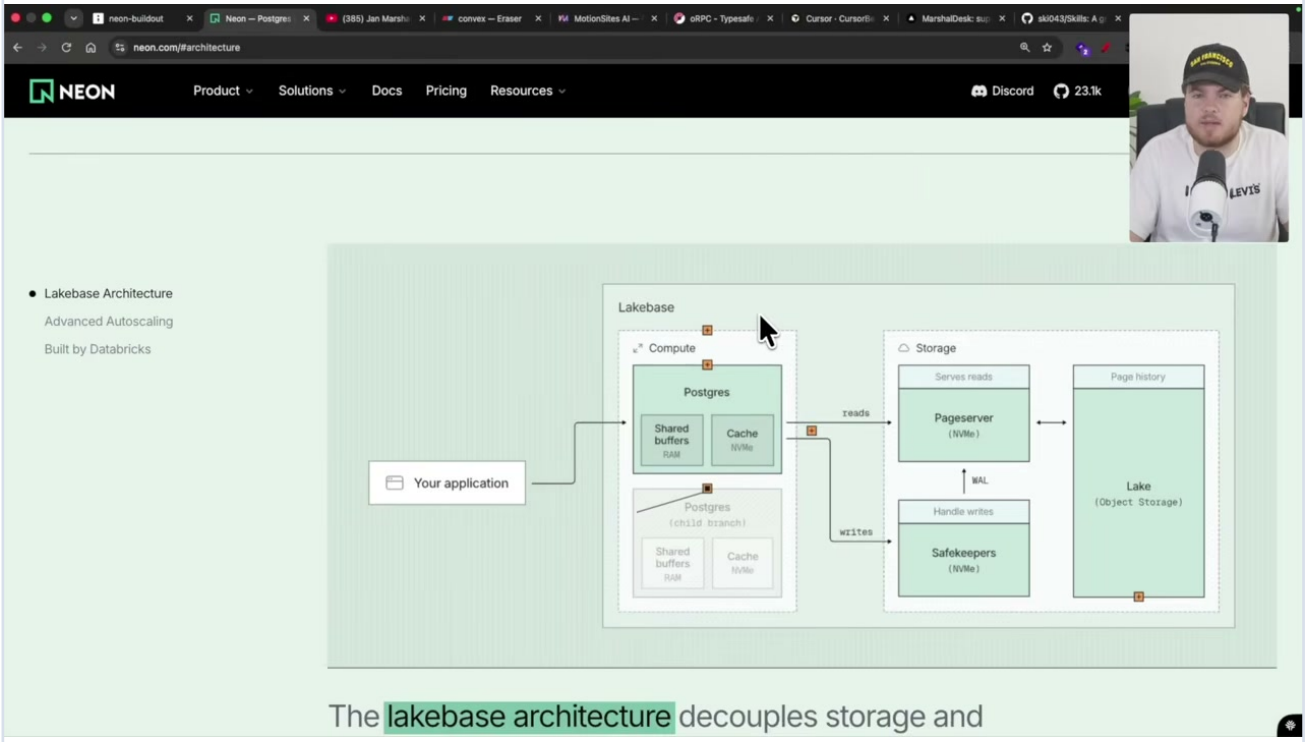
WHY IT MATTERS

Separate thinking from execution and give each role enough fresh context to do one job well.

Source: https://www.youtube.com/watch?v=H84vfy96m_k

Building an Awwwards-Style Landing Page

Let the public page establish the design system



SPEAKER NOTES

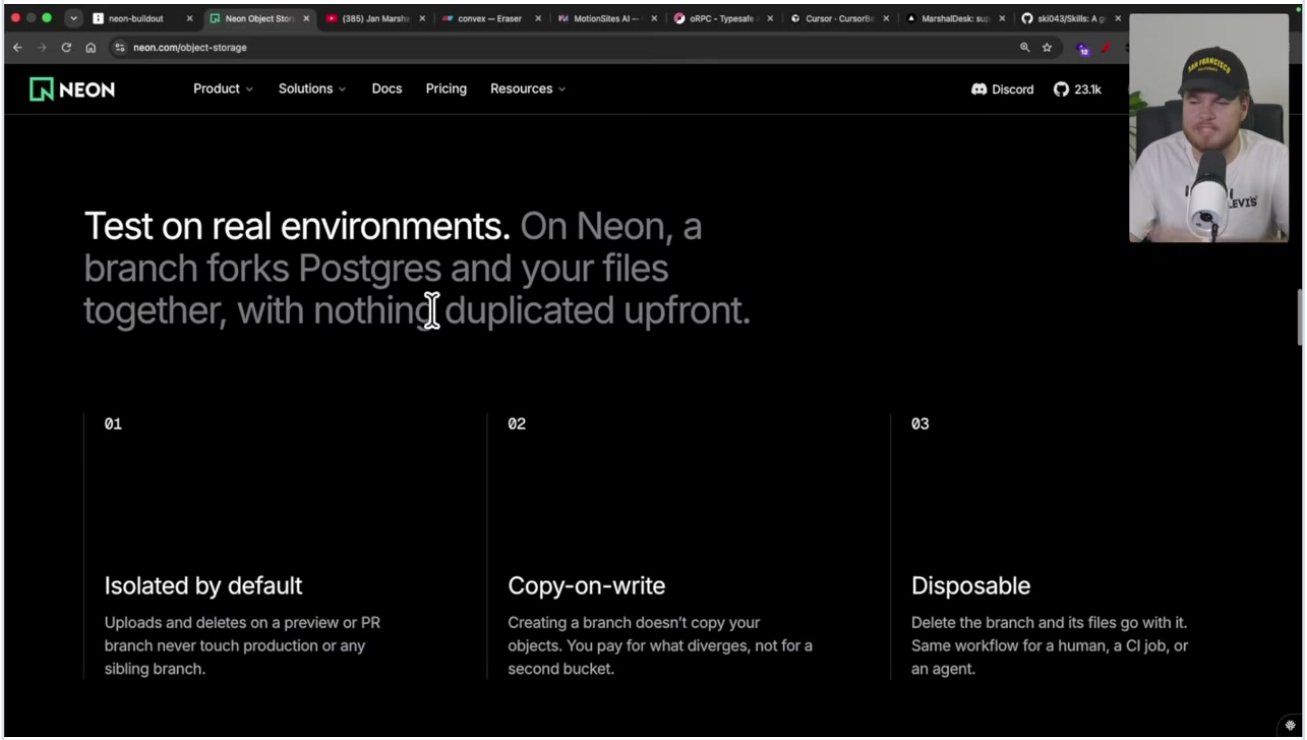
- The landing page is built before authentication and dashboard UI so it can establish typography, color, motion, and component conventions for the product.
- A detailed visual prompt is adapted to the actual Next.js, Tailwind, and shadcn/ui stack rather than copied as static HTML and JavaScript.
- The agent must replace generic copy with product-specific messaging and document the result in DESIGN.md.
- Existing shadcn/ui primitives are extended instead of creating unrelated custom controls.

WHY IT MATTERS

A documented design foundation prevents every later agent from inventing a different visual language.

Evolving the Landing Page

The first render is a prototype



SPEAKER NOTES

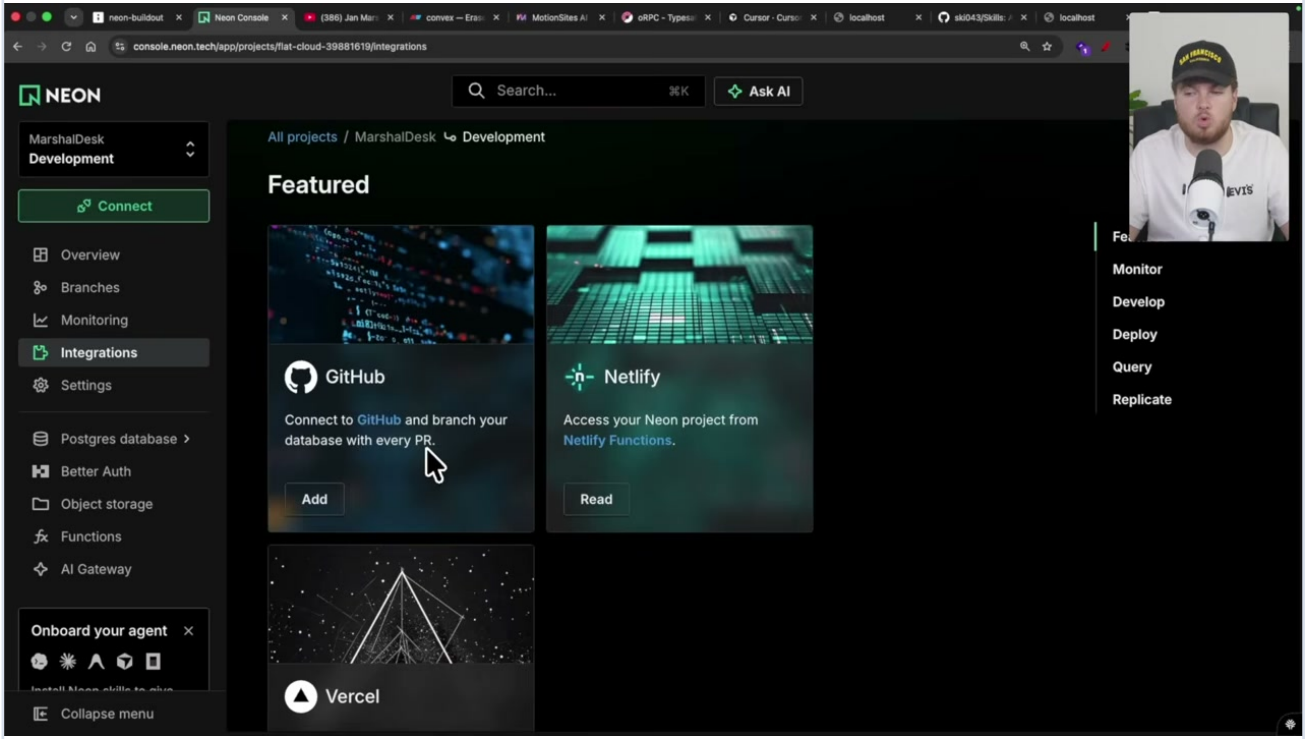
- The generated page is inspected in the browser and refined with element-level feedback rather than accepted because it looks polished at first glance.
- Typography is standardized on Geist Pixel and Geist, and the headline is rewritten around the customer's benefit rather than an abstract agent trait.
- Responsive behavior, menu interaction, video loading, and animations are tested continuously, including on a mobile viewport.
- External fonts and media dependencies are reviewed explicitly instead of passing unnoticed into the product.

WHY IT MATTERS

Browser critique turns a visually plausible draft into a coherent, benefit-driven, responsive design.

Authentication with Better Auth

OAuth, magic links, organizations, and tenancy



SPEAKER NOTES

- Before coding, the workflow verifies Neon skills, MCP access, branch configuration, and the current managed Better Auth integration.
- V1 supports Google OAuth, email/password, and magic links; magic links are restricted to existing users and signup requires email verification.
- The workspace creator becomes the organization owner, while member invitations and full account settings are deferred.
- Development conveniences such as shared OAuth and email credentials must be replaced with dedicated production configuration.

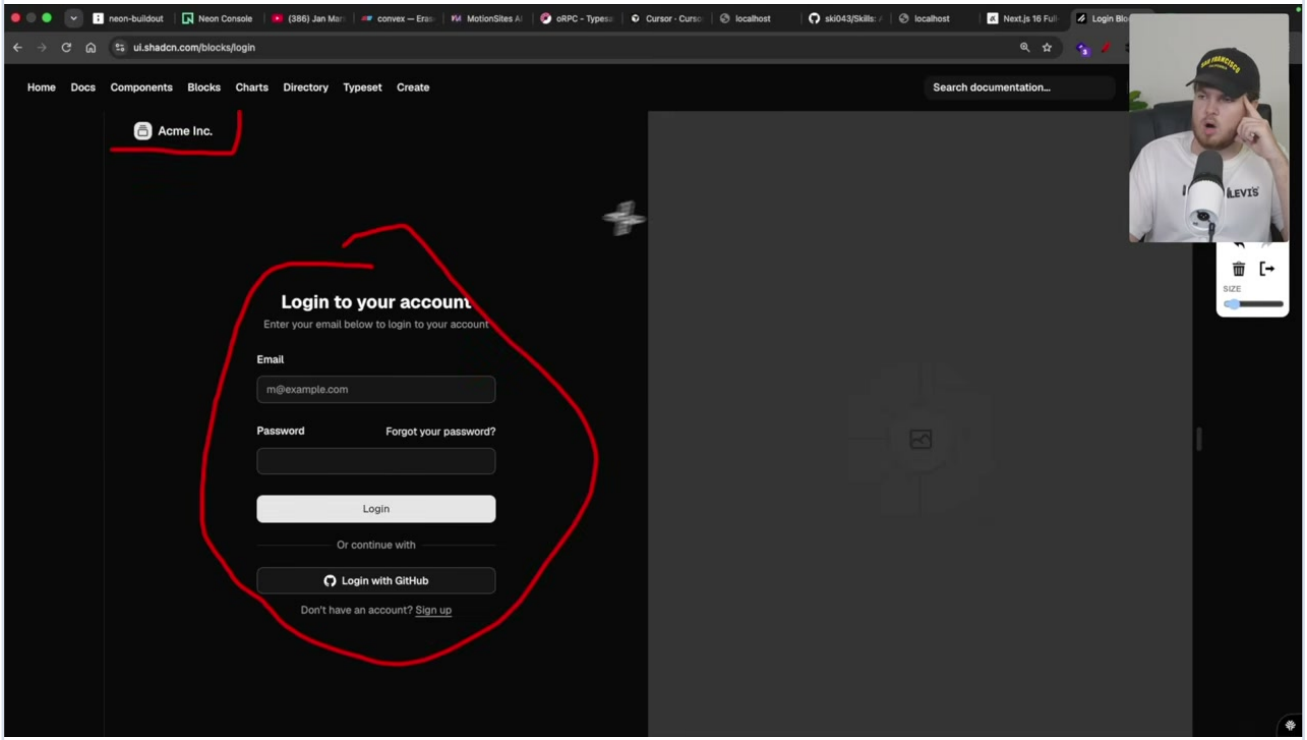
WHY IT MATTERS

Identity behavior and tenant ownership must be settled before code because small ambiguities become security and data-model problems.

Source: https://www.youtube.com/watch?v=H84vfy96m_k

Building the Auth & Dashboard UI

Use proven blocks and framework layouts



SPEAKER NOTES

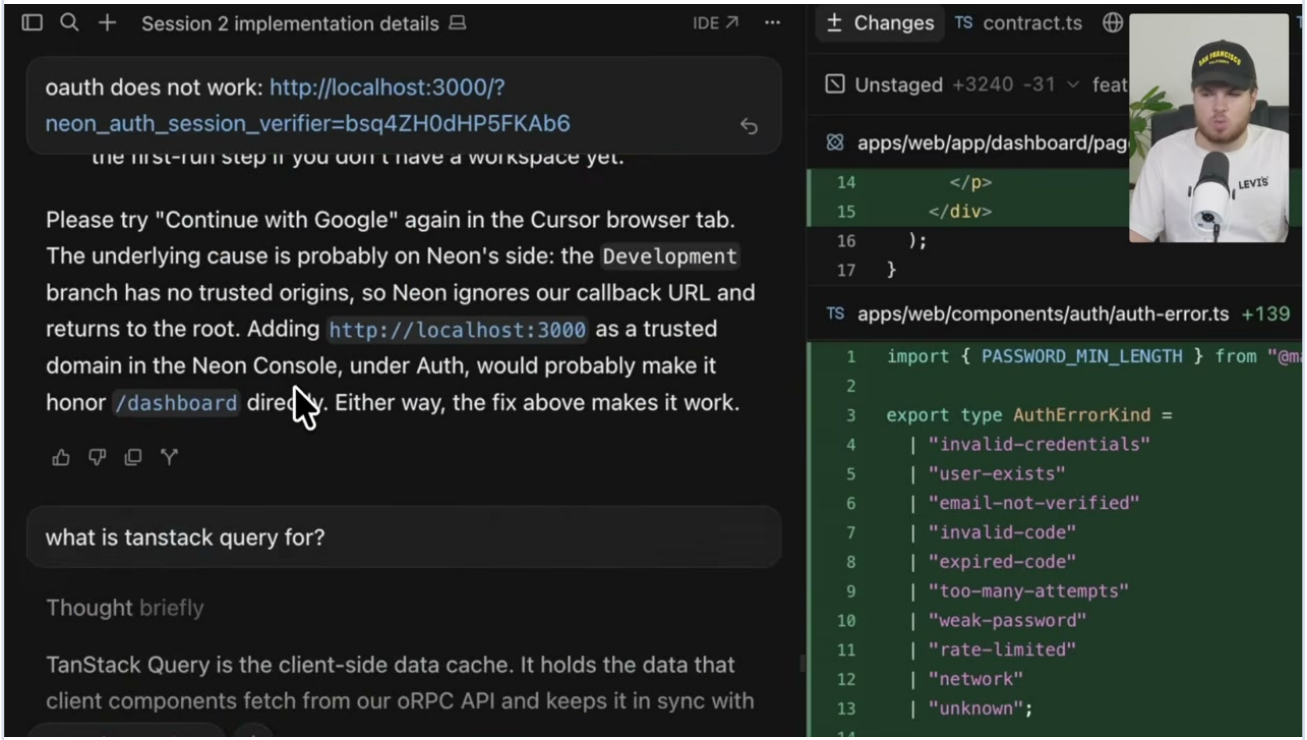
- The implementation combines the auth backend, login/signup UI, and a narrow dashboard shell while leaving dashboard business functionality for later.
- shadcn/ui auth and dashboard blocks reduce avoidable review work around spacing, typography, and standard interactions.
- A shared Next.js auth layout keeps the logo and media panel stable while only the form changes between sign-in and sign-up.
- The dashboard shell is reduced to the actual V1 needs: Home, Inbox, branding, and a user-profile control.

WHY IT MATTERS

Start from stable primitives, then spend review effort on product-specific architecture and behavior.

Code Review Basics

Open the diff and inspect the structure



SPEAKER NOTES

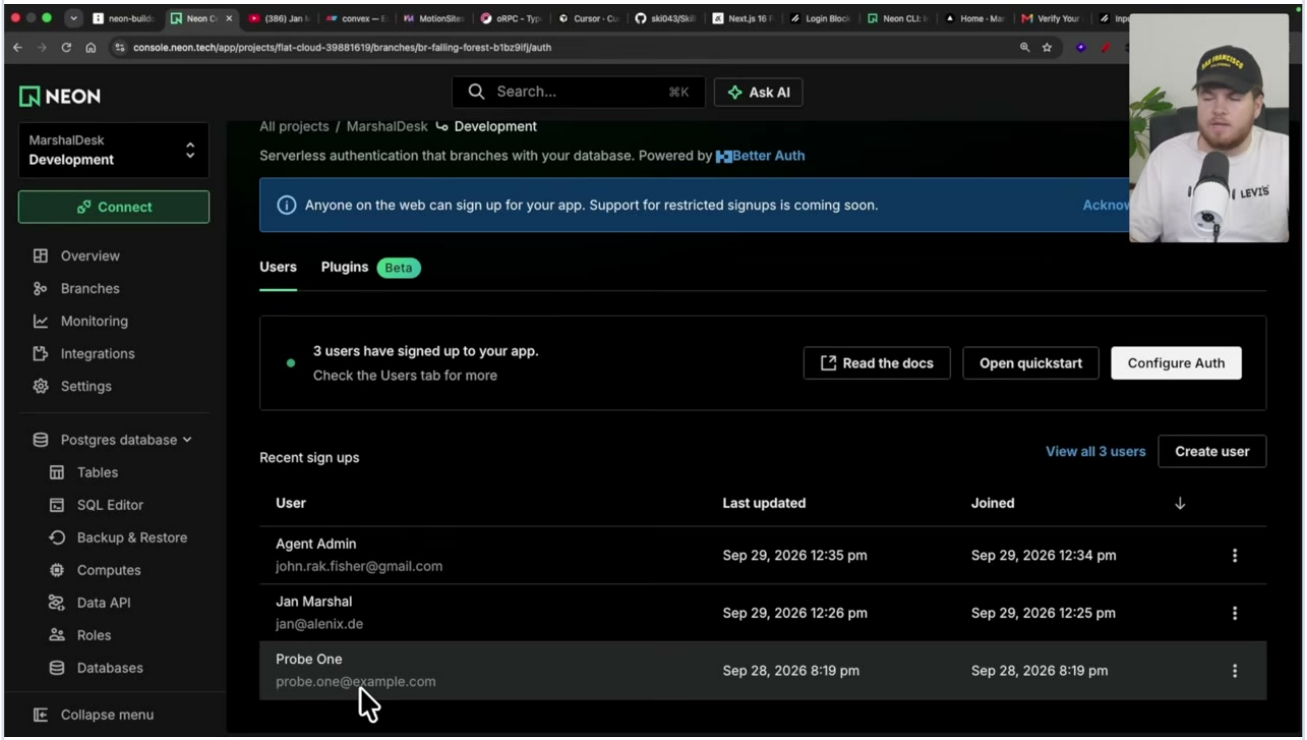
- Modern agents reduce routine review burden, but the engineer still opens the complete diff and reads the changes.
- Human review focuses on duplication, framework misuse, missing route groups, incorrect primitives, and deviations from the agreed architecture.
- The implementation prompt itself is reviewed for repository state, authorization limits, source documents, role routing, and browser-verification requirements.
- Formal plan mode is used selectively; precise context and layered checkpoints may be enough for well-understood personal work.

WHY IT MATTERS

Agent review can catch defects, but only the human owns architectural coherence and acceptance of the change.

Refining the Auth & Dashboard UI

Test the actual user journey



SPEAKER NOTES

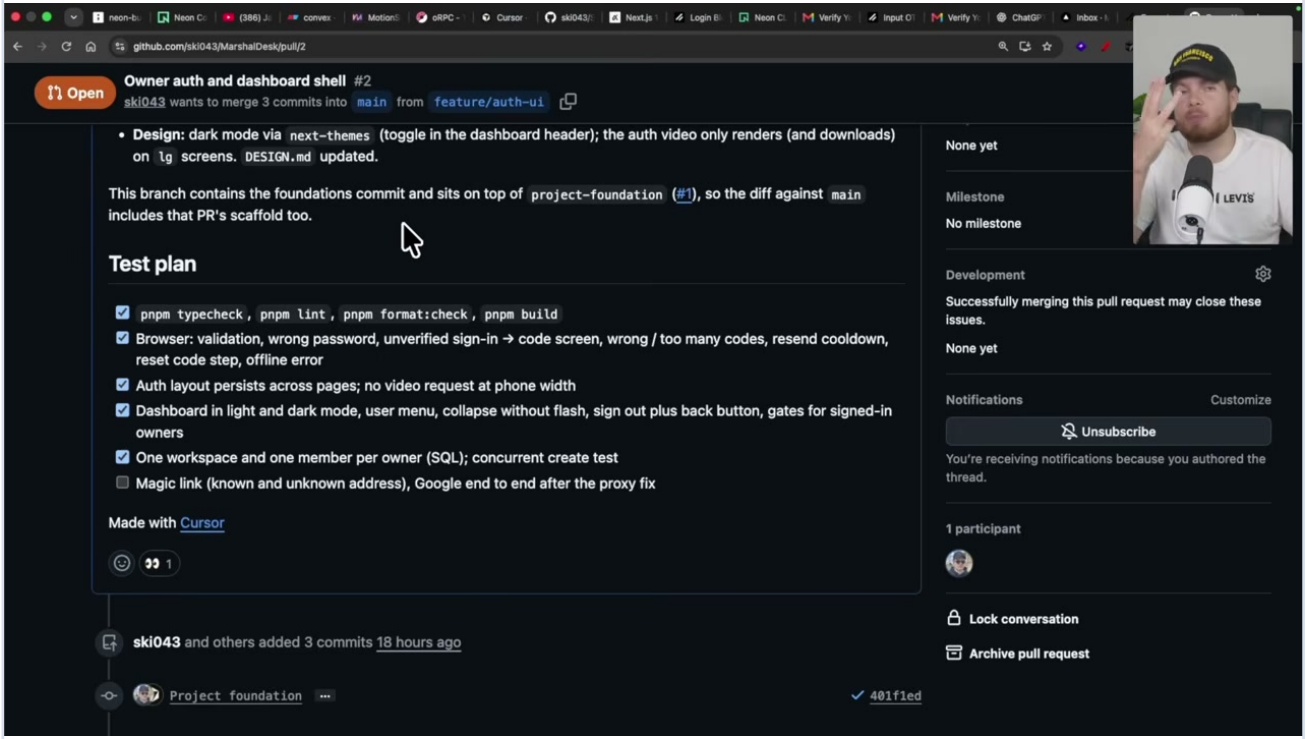
- A successful Google login still loops to the sign-in page, so the callback and session verification path are traced and corrected.
- Equivalent forms are compared: sign-in initially lacks the minimum-password validation that signup already enforces.
- A custom verification input is replaced with shadcn/ui's OTP component, and email verification is tested end to end.
- A preverified test account lets the agent inspect protected pages and correlate the browser state with Neon records.

WHY IT MATTERS

Build success is not journey success; verify redirects, validation, email, protected routes, and persisted identity data.

My 3-Step Code Review Process

Agent review, human review, independent PR review



SPEAKER NOTES

- The lead agent asks reviewer subagents to inspect implementation work before it is handed back.
- The engineer then performs a manual pass over the diff, emphasizing intent, structure, and maintainability.
- An independent PR reviewer such as Cursor Bugbot, Greptile, CodeRabbit, or Pullfrog adds a third perspective.
- Automated comments are not blindly applied: the coding agent first checks whether each finding is valid, then fixes it.

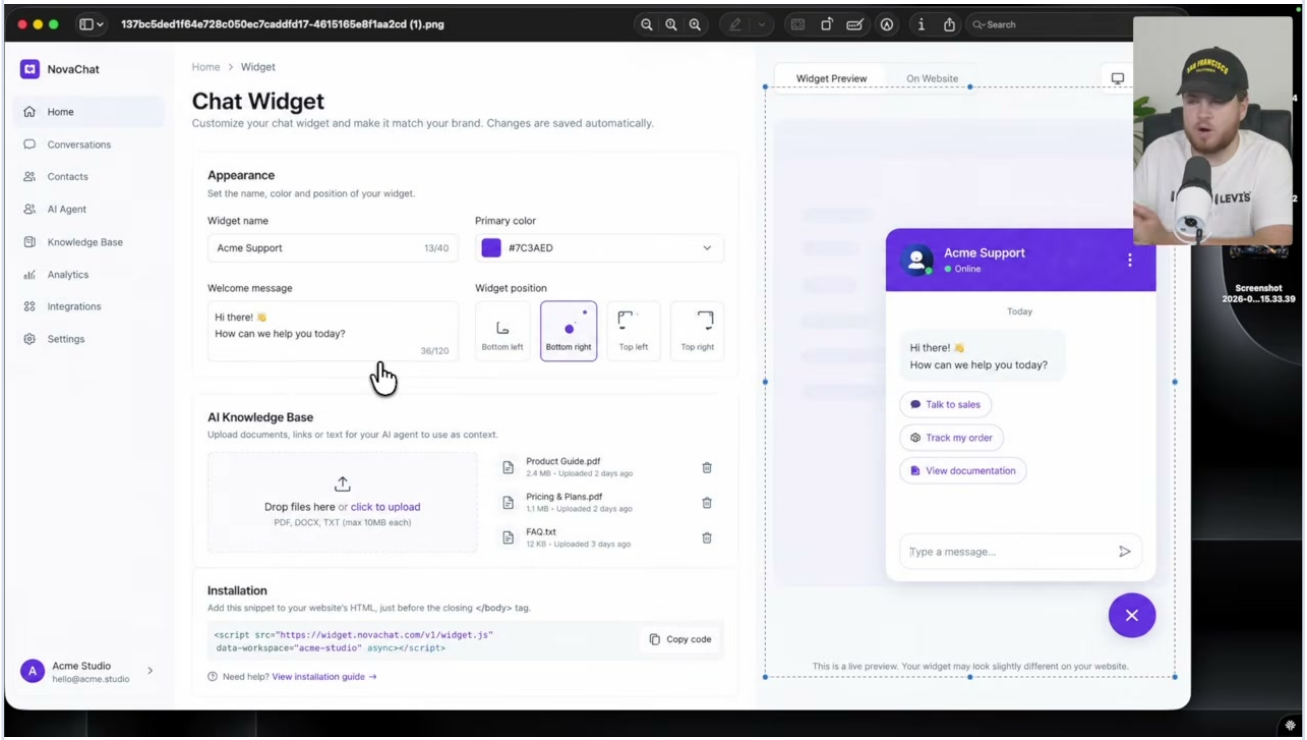
WHY IT MATTERS

Layered reviews catch different classes of failure and reduce the chance that shared assumptions survive to main.

Source: https://www.youtube.com/watch?v=H84vfy96m_k

Gathering Design References

Show the agent what quality means



SPEAKER NOTES

- The widget settings and Inbox are designed from concrete references rather than vague requests to make them premium or modern.
- The settings page uses a configuration/preview split; the Inbox follows a three-column mail-client structure.
- References communicate composition and hierarchy, but they are adapted to the established Marshal Desk design system rather than copied.
- Conversation selection is placed in the URL so refreshes and shared links preserve the active thread.

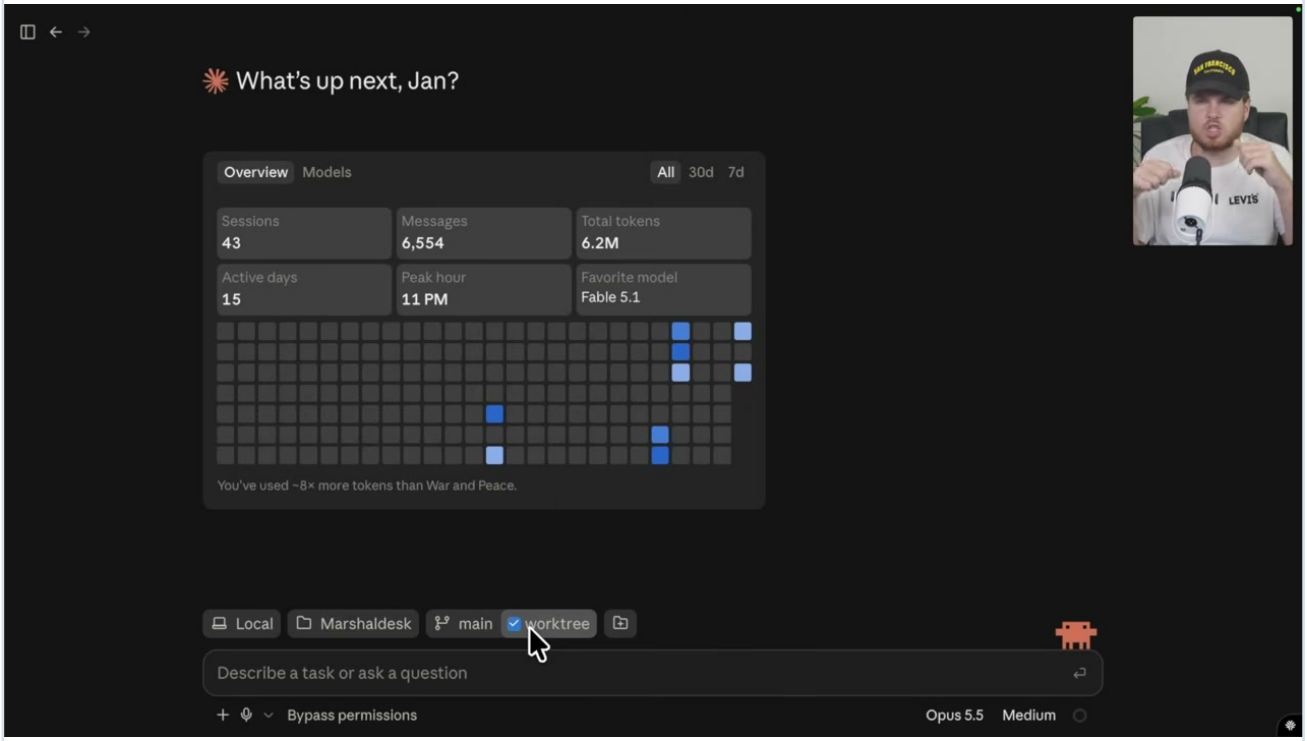
WHY IT MATTERS

A visual reference conveys layout intent more accurately than aesthetic adjectives.

Source: https://www.youtube.com/watch?v=H84vfy96m_k

Running Multiple Agent Sessions in Parallel

Parallelize by ownership boundary



SPEAKER NOTES

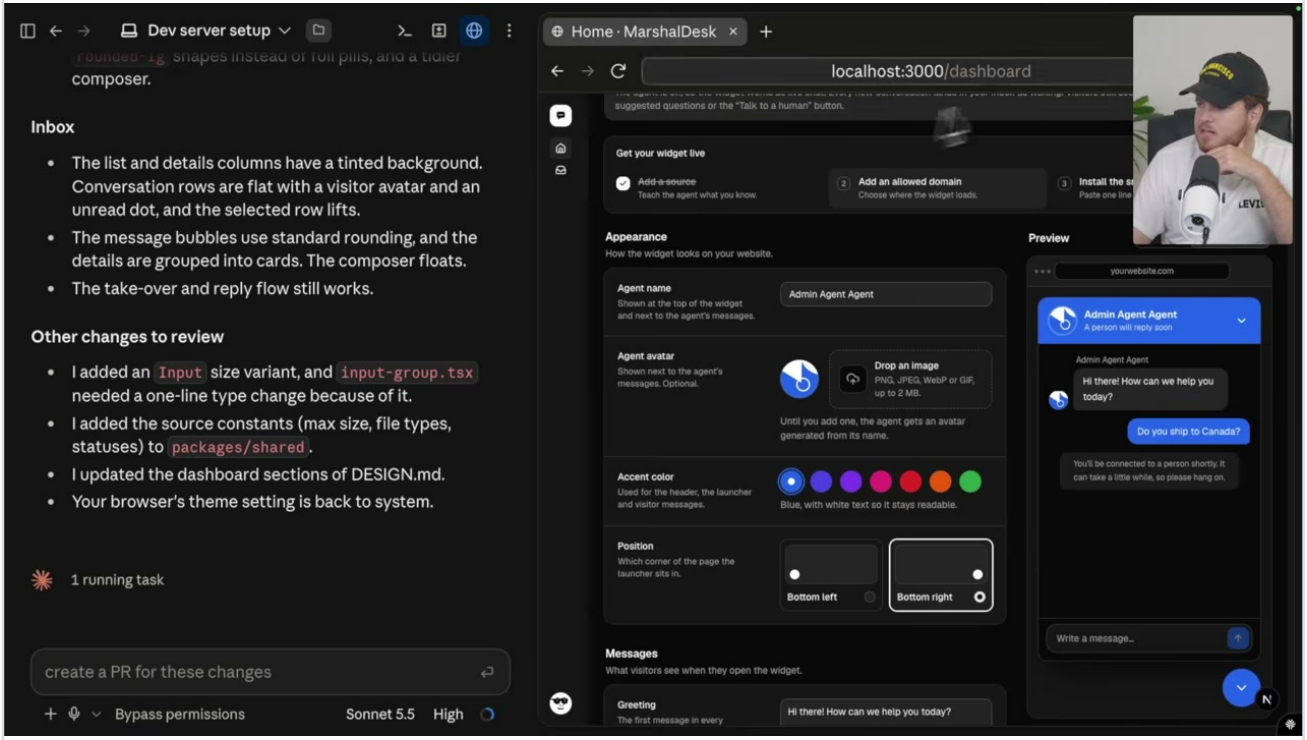
- A worktree isolates the visual-refinement session from other changes while separate implementers own the widget route and Inbox route.
- Shared dependencies and common shadcn/ui components are installed before parallel work begins to prevent package and foundation conflicts.
- The lead session remains responsible for integration while implementers work only within distinct route boundaries.
- Type checks and linting are necessary but do not replace authenticated browser inspection and visual review.

WHY IT MATTERS

Parallel work accelerates delivery only when shared foundations are stabilized and file ownership is clear.

Using Design Mode in Claude Code & Cursor

Iterative element-level critique



SPEAKER NOTES

- Design Mode is treated as a critique interface: select the rendered element, preserve what works, and attach precise feedback to what does not.
- Iterations correct pill-shaped controls, placeholders, password visibility, divider backgrounds, OTP styling, and exaggerated hover growth.
- Owned shadcn/ui components should contain their variants rather than exporting page-level class constants.
- Automated PR review later catches iOS input zoom and OTP overflow—issues that a desktop visual pass can miss.

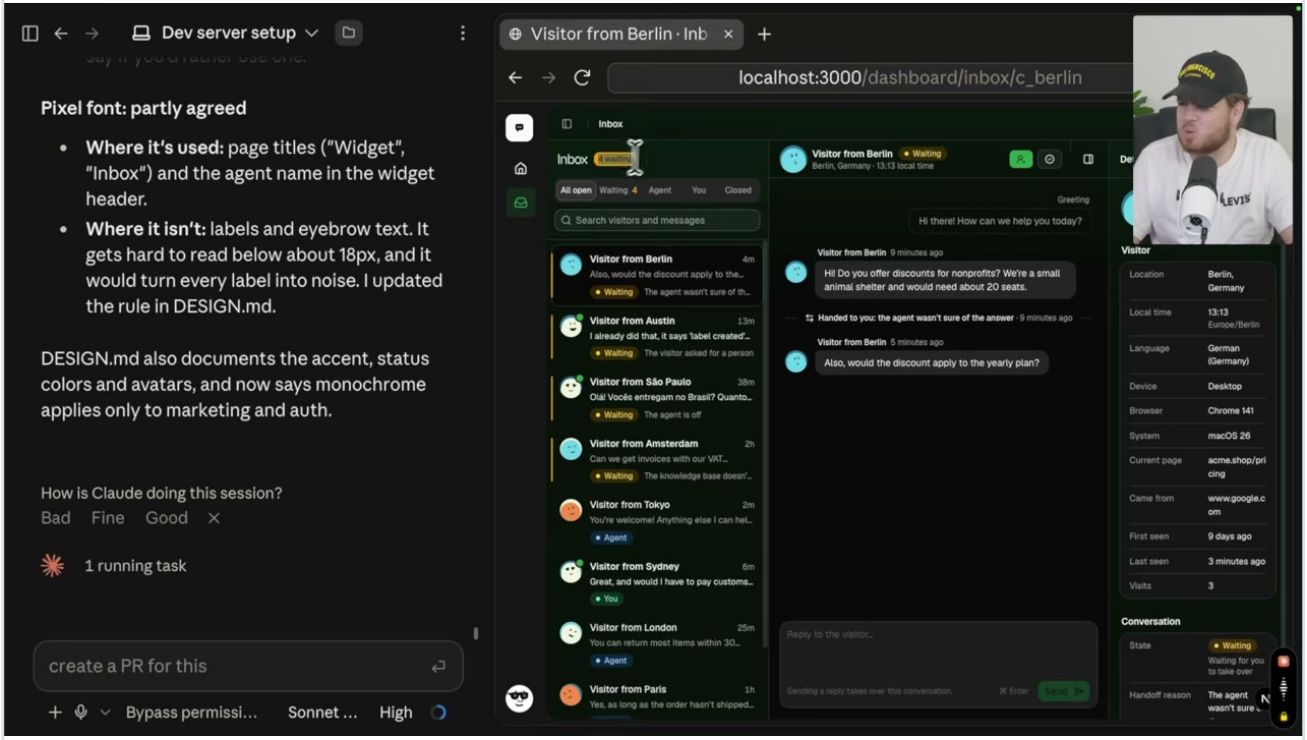
WHY IT MATTERS

High-quality interface work emerges from repeated observation and correction, not a one-shot design prompt.

Source: https://www.youtube.com/watch?v=H84vfy96m_k

Fighting AI Slop & Fixing Issues

Remove decoration without purpose



SPEAKER NOTES

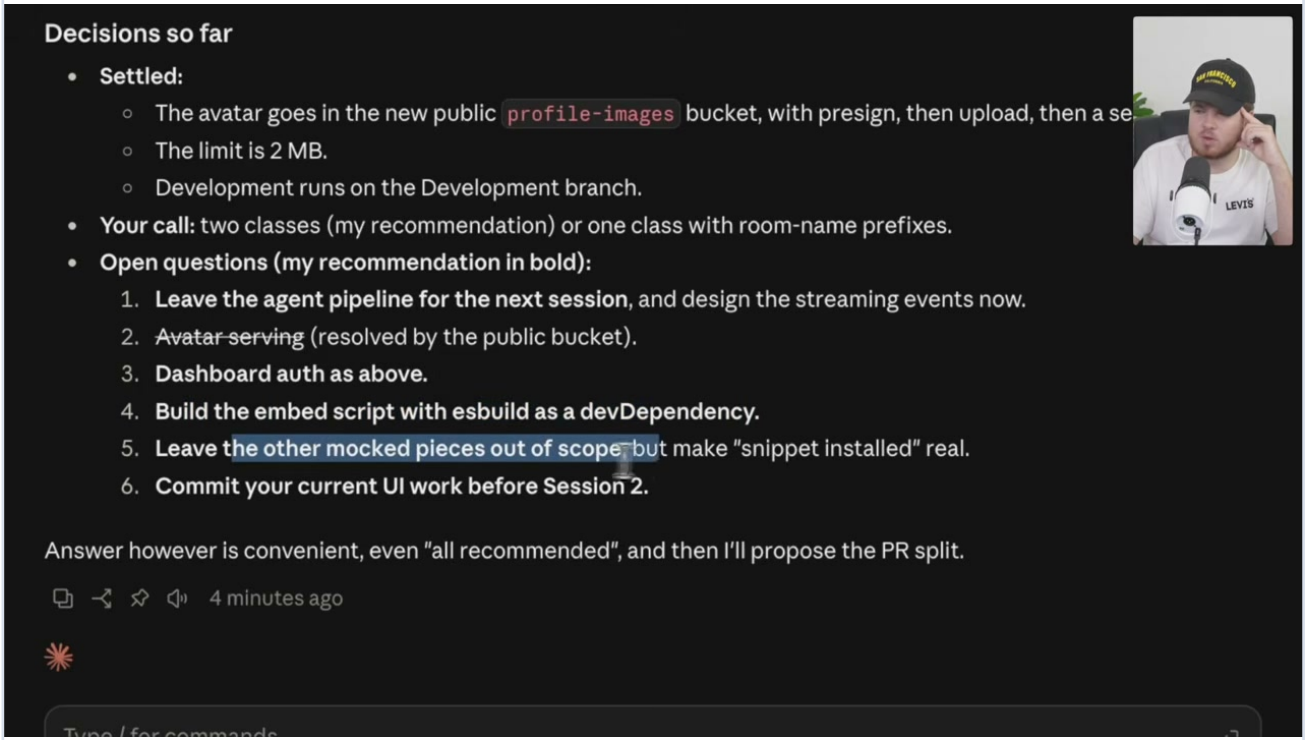
- The agent spreads accent colors across large inbox surfaces, producing arbitrary green and orange treatments that conflict with the product.
- The correction prompt preserves the useful top glow and pixel label while restoring neutral structural surfaces.
- Color is restricted to meaningful state, workspace accent, and status communication rather than becoming ambient decoration.
- A remaining nonblocking selector issue is deferred so visual perfection does not delay functional backend work.

WHY IT MATTERS

Name the failed choices, preserve what worked, and revise against an approved screen.

Backend: Database, Real-Time Chat & More

Connect the established UI to real systems



SPEAKER NOTES

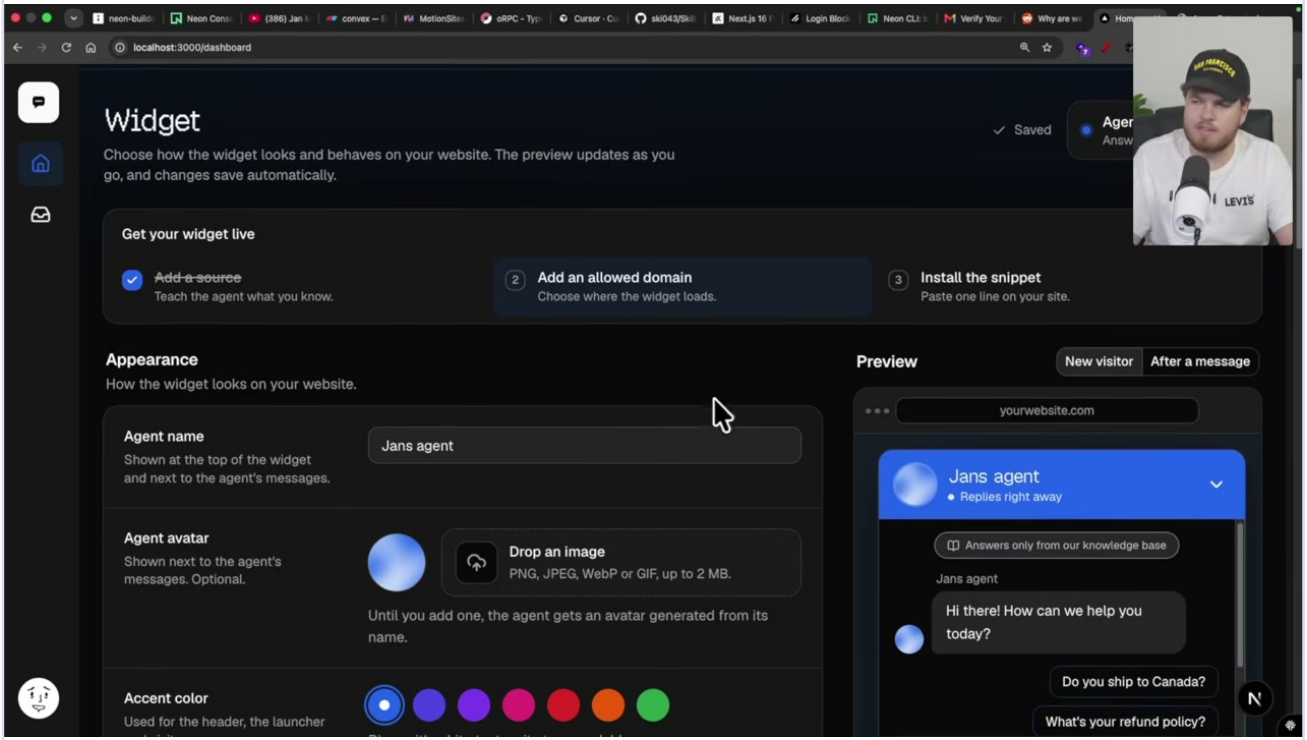
- The backend increment persists widget settings, uploads avatars to object storage, produces an embeddable iframe loader, and adds PartyServer real-time messaging.
- Public avatars and private knowledge files use different storage policies; presigned uploads remain server-authorized and size-limited.
- Conversation rooms synchronize one chat, while workspace rooms notify the owner about new conversations and aggregate state.
- Short-lived WebSocket tokens are issued through an authenticated procedure because browser sockets cannot attach arbitrary authorization headers.

WHY IT MATTERS

Settle storage, tenancy, room topology, authentication, and failure behavior before connecting a large vertical slice.

Testing the Full Flow

Validate behavior across browser, real time, and database



SPEAKER NOTES

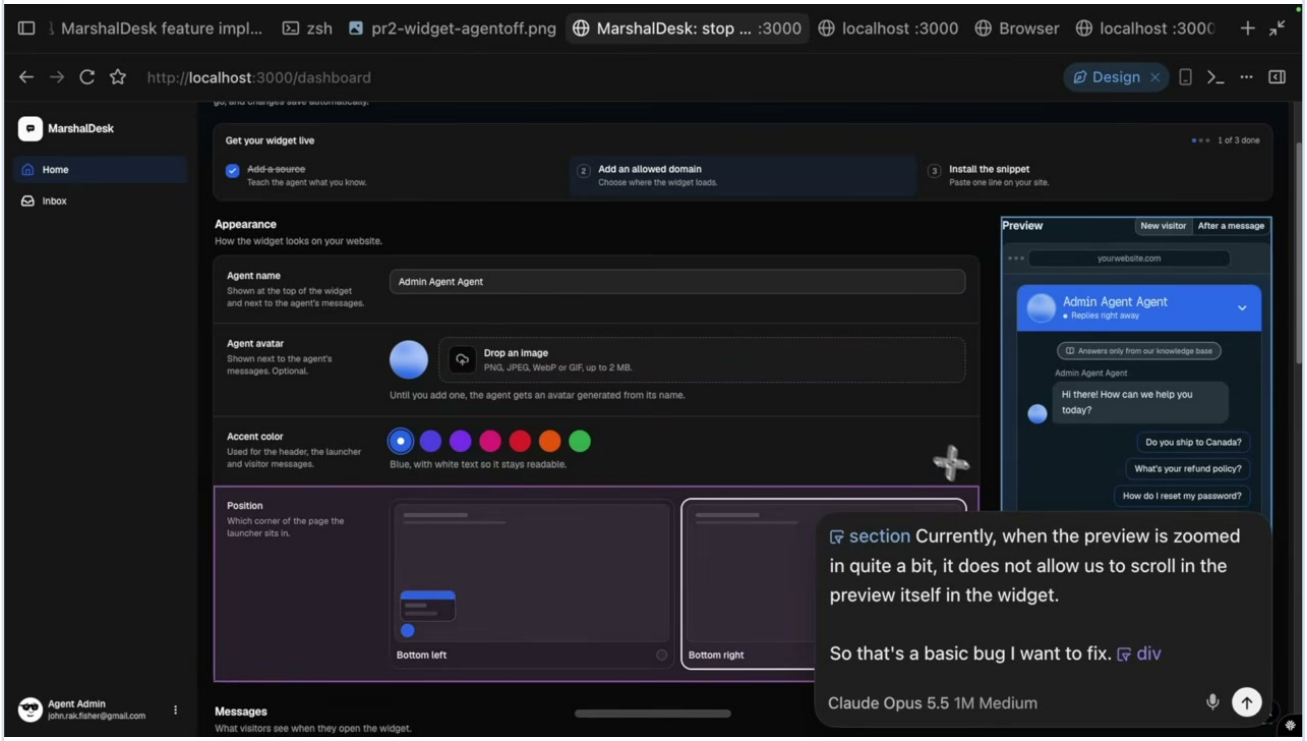
- Autosave is tested with a hard refresh; live in-memory changes alone do not prove persistence.
- Allowed-domain enforcement, embed loading, visitor message creation, owner replies, notifications, typing indicators, and real-time configuration updates are tested separately.
- Conversation closure, restart, and human-request states are exercised instead of stopping at the happy path.
- Neon tables and monitoring verify that conversations, messages, visitors, workspaces, and members exist as expected.

WHY IT MATTERS

A convincing demo is not enough—prove browser behavior, authorization, real-time state, lifecycle transitions, and infrastructure effects.

Refining the Result

Review before beginning the next feature



SPEAKER NOTES

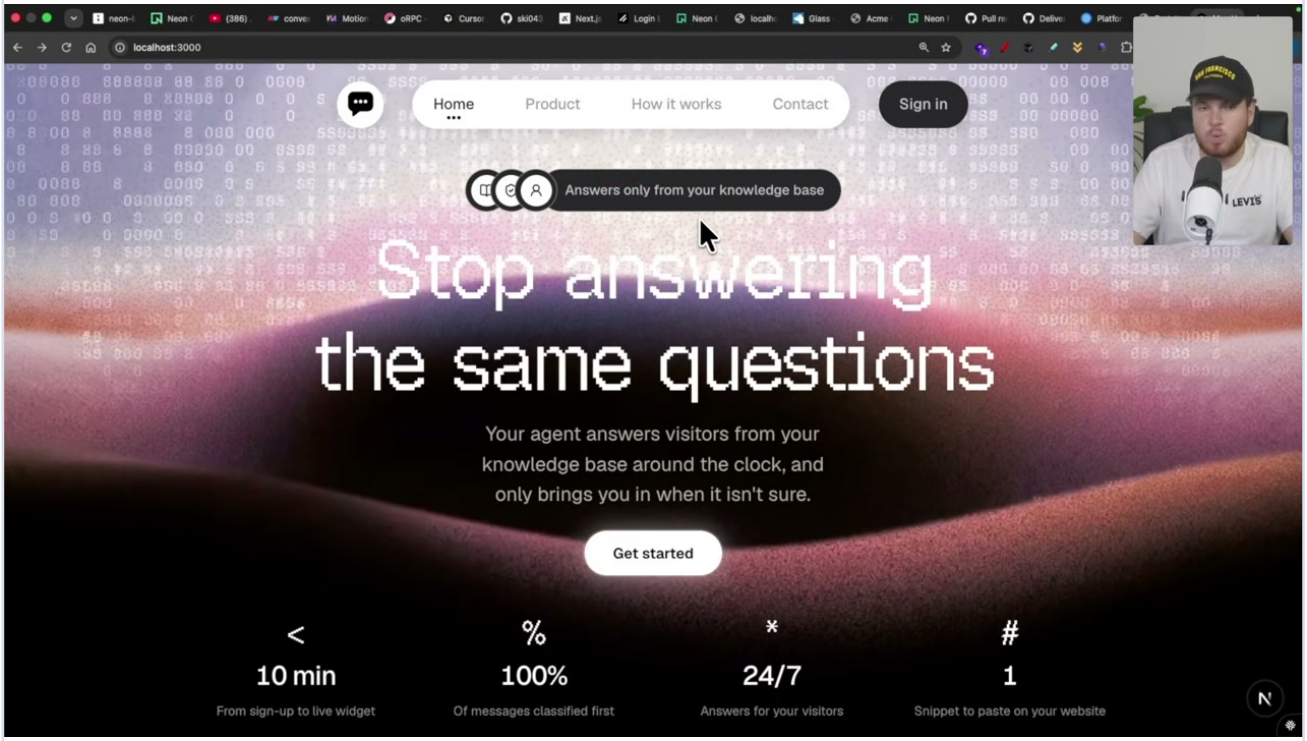
- The application is checked against Neon as an independent source of truth before the team moves on to the AI feature.
- A human review of roughly 35,000 generated lines removes unnecessary error-handling abstractions and builds understanding of the codebase.
- Bugbot then finds a personal test-host path and a stale-list mutation issue missed by the manual pass.
- A small account-button fix loads session state on the client to preserve static generation for the public homepage.

WHY IT MATTERS

Working output should trigger review and cleanup, not an immediate jump to the next feature.

Building the AI Agent (RAG Pipeline)

Separate ingestion from answering



SPEAKER NOTES

- Document ingestion runs in Neon Functions: extract PDF/Markdown/TXT, chunk text, create Qwen 3 embeddings, and store workspace-scoped vectors in PostgreSQL/pgvector.
- After a visitor message is saved, Next.js continues processing: classify the message, embed the query, retrieve the top chunks, and apply a similarity threshold.
- Insufficient evidence or explicit human requests cause handoff; grounded answers stream through PartyServer and are saved before completion is broadcast.
- Candidate models are benchmarked against the real gateway; latency and structured behavior matter more than nominal price or reputation.

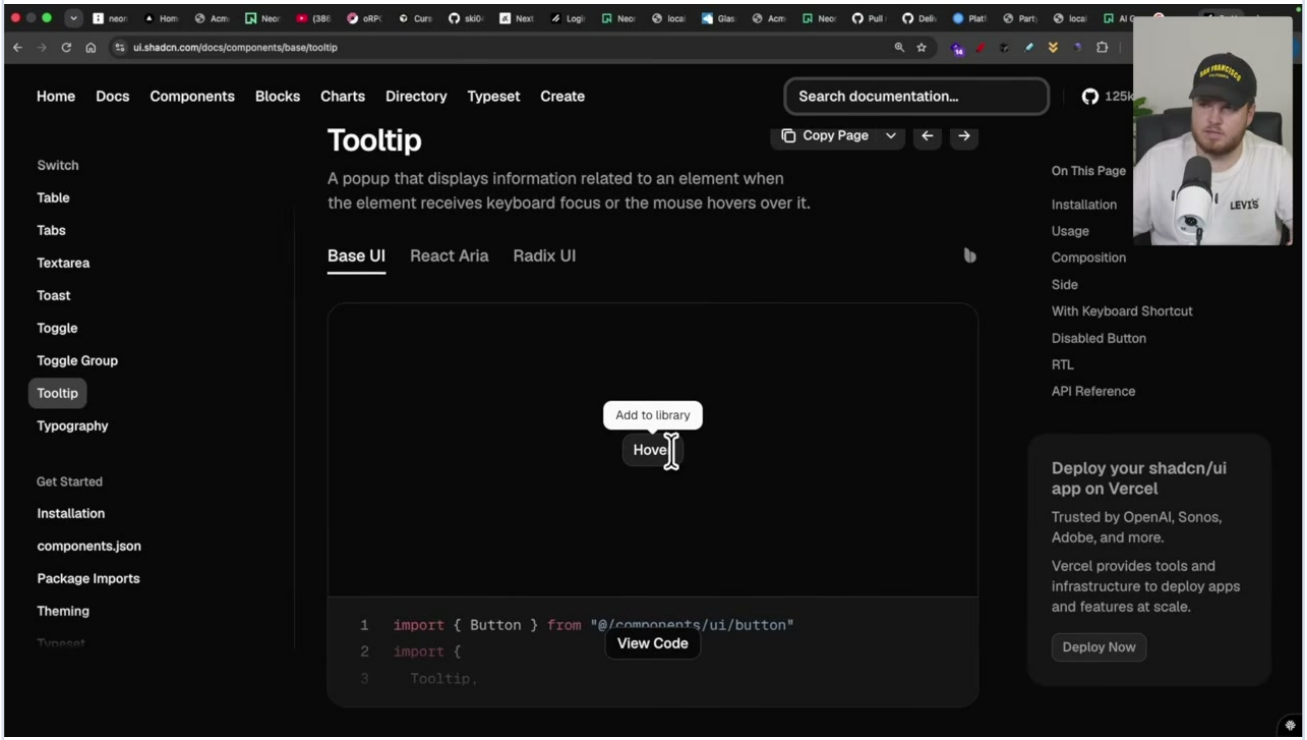
WHY IT MATTERS

Write the complete RAG control flow and conservative fallback behavior before letting an agent implement it.

Source: https://www.youtube.com/watch?v=H84vfy96m_k

Testing & Reviewing the AI Agent

Grounding, provenance, handoff, and observability



SPEAKER NOTES

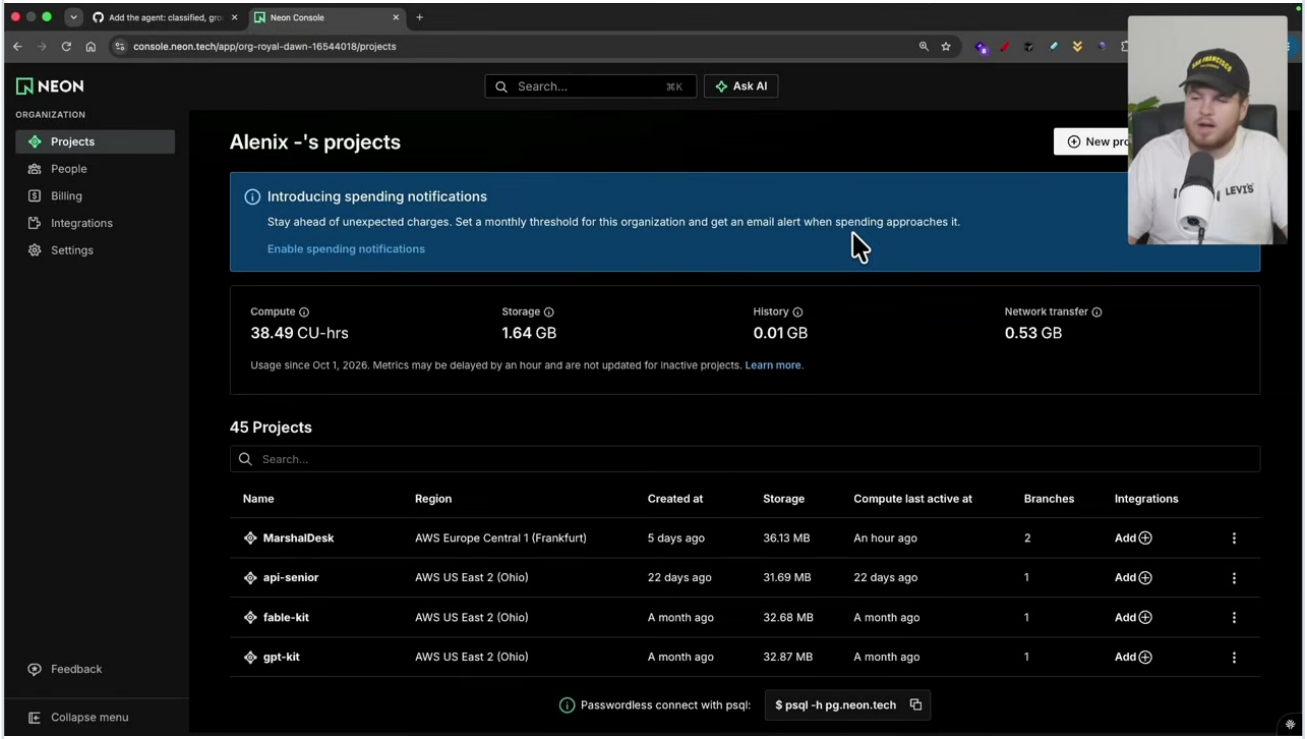
- Several knowledge sources are uploaded in parallel and verified through function logs, object storage, database state, and resource metrics.
- Tests include answerable questions, undocumented discounts, refusal to complete verification, prompt injection, explicit handoff, and retrieval misses.
- Operator-facing replies list source documents and open a chunk viewer, while visitor-facing text avoids exposing internal retrieval details.
- The UI is refined for wrapping, contrast, tooltips, file-type icons, message distinction, and restrained motion with reduced-motion preserved.

WHY IT MATTERS

Test grounding, policy boundaries, evidence, and safe failure paths—not fluency alone.

Finding & Fixing Issues

Three pre-deployment passes



SPEAKER NOTES

- A reviewer discovers a prompt-injection path where visitor text can imitate a team member; structured history and trusted author metadata close the hole.
- Behavioral testing covers normal, adversarial, unsupported, and human-handoff scenarios while operational testing covers logs, metrics, storage, and performance.
- Large stacked PRs are reviewed for duplicate helpers, unsafe casts, stale framework patterns, and generic error wrappers that discard oRPC's typed errors.
- Independent PR review finds a stale stream after switching conversations, demonstrating that every layer catches different defects.

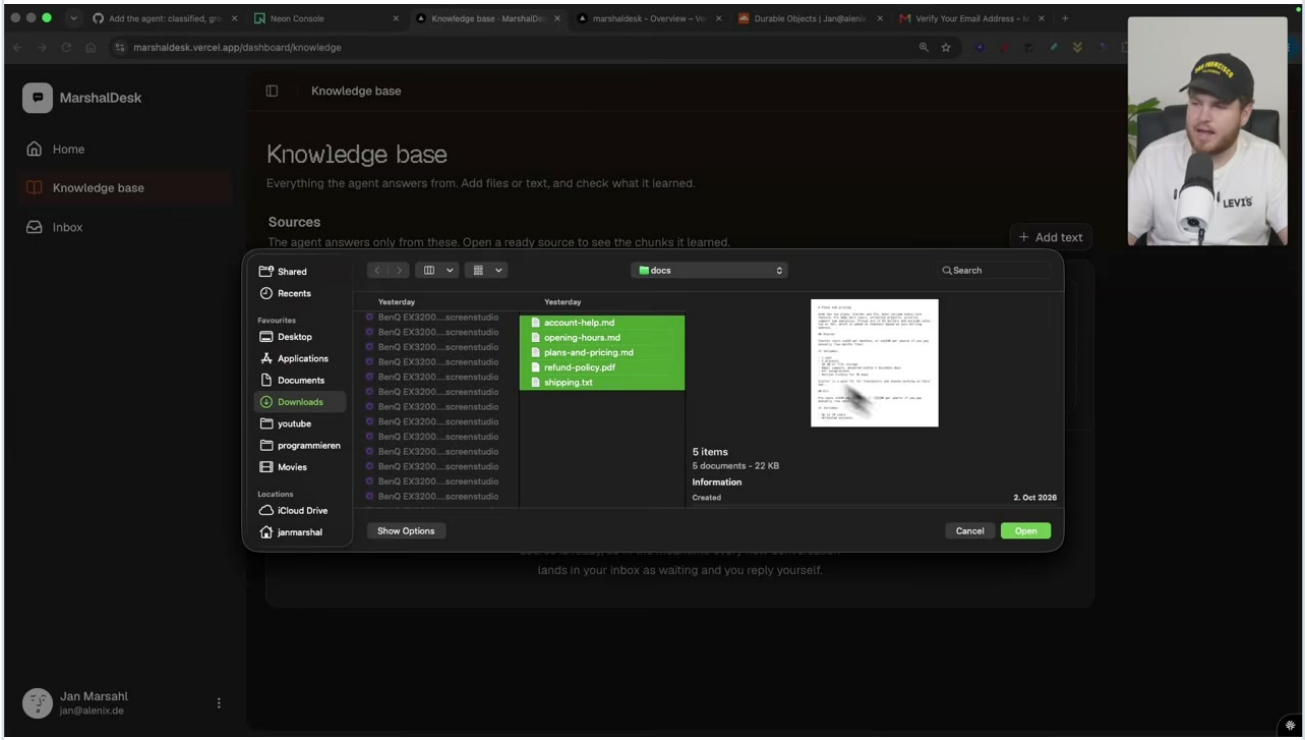
WHY IT MATTERS

Before deployment, run separate behavioral, operational/UX, and code-quality passes.

Source: https://www.youtube.com/watch?v=H84vfy96m_k

Deploying to Production

An ordered migration with real smoke tests



SPEAKER NOTES

- Production spans an isolated Neon branch, a Cloudflare PartyServer deployment, and a Vercel application; each receives its own data, secrets, storage, functions, and trusted origins.
- The deployment order starts with the production URL, then secrets, migrations, buckets, AI credentials, CORS, workers, background jobs, Vercel configuration, and authentication.
- MCP integrations perform mechanical provisioning, but the developer retains decisions about domains, OAuth strategy, plans, approvals, and launch acceptance.
- A real signup, autosave, knowledge upload/delete/re-upload, grounded answer, human handoff, notifications, typing indicators, source inspection, and conversation closure are all tested in production.

WHY IT MATTERS

A green deployment is not completion; independently inspect the infrastructure and prove the full workflow as a real user.